命令行入门：操作目录、文件和应用

平时我们用鼠标打开文件夹、复制文件、启动应用。命令行让我们用文字完成这些操作：告诉电脑“到哪里去”“处理哪个文件”“打开哪个项目”。学会这些基本动作，后面运行程序和使用 Git 就有了起点。

本文在 **Windows 的 PowerShell** 中操作，使用 `pwd`、`ls`、`cd`、`mkdir`、`cat`、`cp`、`mv`、`rm` 等短命令。按文中的选项和示例输入即可。后半部分用一个练习项目介绍 Git 的基本操作。

1. 先弄清楚：文件放在哪里，我现在在哪里？

在文件资源管理器中，你能看到自己打开了哪个文件夹。到了命令行里，也要先确认位置，否则同样一句“打开 README.txt”，可能打开另一个文件，或者根本找不到文件。

1.1 目录就是文件夹，路径就是地址

要告诉电脑操作哪个文件，先得描述它的位置。可以把目录理解为一层层文件夹，把路径理解为找到文件的地址。

假设有下面这些文件和文件夹，`student` 是示例用户名：

```
C:\
└── Users\
    └── student\
        ├── cli-practice\
        │   ├── README.txt
        │   ├── notes\
        │   │   └── lesson.txt
        │   └── images\
        └── Downloads\
```

- `C:` 是盘符，`C:\` 是 C 盘的根目录，也就是这一盘目录的最上层。
- `cli-practice`、`notes`、`images` 都是目录，也就是文件夹。
- `README.txt`、`lesson.txt` 是文件。
- `notes` 放在 `cli-practice` 里面，所以 `cli-practice` 是它的**父目录**，`notes` 是 `cli-practice` 的**子目录**。
- `notes` 和 `images` 放在同一层，可以把它们理解为两个“相邻的房间”。

`C:\Users\student\cli-practice\notes\lesson.txt` 是文件的完整地址：从 C 盘开始，依次打开 `Users`、`student`、`cli-practice`、`notes`，最后找到 `lesson.txt`。

1.2 用 `pwd` 看位置，用 `ls` 看周围

打开命令行后，第一件事是确认自己“站在哪个文件夹里”，再看看这里有什么。

在开始菜单搜索并打开 PowerShell。如果打开的是 Windows Terminal，选择其中的 PowerShell 标签页。窗口里的提示符通常类似：

```
PS C:\Users\student>
```

其中的路径提示当前位置。输入下面的命令，再按 Enter，也能查看当前位置：

```
pwd
```

接着列出这个目录中的文件和子目录：

```
ls
```

当前位置，也叫当前工作目录，就是接下来操作的出发点。 命令行不会猜测你想操作桌面、下载目录还是项目目录，需要你先选对位置。

文中的命令块只包含需要输入的内容。教程里如果出现 `PS C:\...>`，它是提示符，不需要跟着输入；命令执行后的输出也不需要复制。

2. 用 `.`、`..` 和 `~` 在目录之间走动

每次操作都写完整地址太麻烦。命令行提供了几个简短的写法，分别表示“这里”“上一层”和“我的主目录”。理解它们后，就能像在文件资源管理器里进入文件夹、返回上一级一样移动。

这一节先用上面的目录树说明。示例目录需要实际存在才能执行；第 4 节会带你从头创建自己的练习目录。

2.1 `cd`：进入一个文件夹

想操作另一个文件夹里的内容，通常先用 `cd` 切换过去。可以把 `cd` 理解成“进入这个目录”。

假设当前位于 `C:\Users\student\cli-practice`：

```
cd notes
pwd
```

执行后，位置变成：

```
C:\Users\student\cli-practice\notes
```

`cd notes` 就像在文件资源管理器里双击 `notes` 文件夹。`cd` 后面应是目录，不能写 `cd lesson.txt` 来打开文件。

2.2 `.`：就是“我现在所在的这个目录”

有时我们需要把当前文件夹交给一个命令处理，又不想重写它的地址。这时用一个点 `.` 就可以表示“这里”。

假设当前位于 `cli-practice`，下面两条命令都会列出这里的内容：

```
ls
ls .
```

在路径开头写 `.\`，就是明确说“从这里开始找”：

```
ls .\notes
cat .\README.txt
```

第一行查看**当前目录下面的** `notes` 里有什么，第二行读取**当前目录下面的** `README.txt`。这两条命令都不会改变当前位置，用 `pwd` 看仍然在 `cli-practice`。

如果想进入这里的 `notes`：

```
cd .\notes
```

它和 `cd notes` 表达的意思一样。反过来，`cd .` 表示进入当前目录，所以位置不会变化。

后面还会用到 `explorer .`，让文件资源管理器打开“我现在所在的文件夹”。**点本身只表示位置，要做什么由前面的命令决定。**

2.3 `..`：上一层，也就是父目录

进入一个文件夹后，要返回包着它的那个文件夹，就用 `cd ..`。这里的两个点 `..` 表示**上一层目录，也叫父目录**，可以理解成“往外退一层”。

假设你现在位于：

```
C:\Users\student\cli-practice\notes
```

执行：

```
cd ..
pwd
```

现在到了 `C:\Users\student\cli-practice`。再执行一次：

```
cd ..
pwd
```

又退了一层，到了 `C:\Users\student`。

也可以一次退两层。下面是一个**独立例子**，先回到 `notes`，再退两层：

```
cd ~\cli-practice\notes
cd ..\..
pwd
```

结果同样是用户主目录。`..\..` 就是“上一层的上一层”；路径中的每一个 `..` 都表示向上走一层。

还可以先退一层，再进入旁边的目录。假设从 `notes` 去同一层的 `images`：

```
cd ~\cli-practice\notes
cd ..\images
pwd
```

走的路线是：

```
notes → cli-practice → images
          上一层          子目录
```

`..` 也可以交给其他命令使用，不一定要跳转：

```
ls ..
cat ..\README.txt
```

如果当前位置是 `images`，第一行查看父目录 `cli-practice` 的内容，第二行读取父目录里的 `README.txt`，**你自己仍然留在 `images`**。

因此，严格地说，`..` 是“上一层的位置”，`cd ..` 才是“跳转到上一层”。它也不是“回到上一次去过的地方”：无论之前去过哪里，`cd ..` 都只按目录层级往上走。

2.4 `~`：一键回到自己的主目录

走进很多层目录后，逐次 `cd ..` 太慢。`~` 表示当前用户的主目录，相当于一个固定的“回家地址”。

在 Windows 上，它通常类似 `C:\Users\你的用户名`。比如示例用户 `student` 的主目录是 `C:\Users\student`。从任何文件夹都可以执行：

```
cd ~
pwd
```

这样就回到自己的主目录。它不是 C 盘根目录，也不是当前项目的根目录。

`~` 后面也可以接路径，从自己的主目录直接找到目标：

```
cd ~\cli-practice
ls ~\Downloads
```

第一行进入主目录下面的练习目录；第二行查看主目录下面的下载目录，不改变当前位置。如果你的下载目录被移动过，应换成实际位置。

区别在于：`.` 和 `..` 会随着当前位置变化，`~` 对同一个用户通常指向固定的主目录。

2.5 完整地址与相对路线怎么选？

要去很远的目录，可以直接给完整地址；处理附近的文件，用相对路线更方便。

写法	怎么理解	适合什么时候用
`C:\Users\student\cli-practice\notes`	从 C 盘开始的完整地址，也叫绝对路径	已经知道目标的完整位置
`.\notes`	从这里进入 `notes`，属于相对路径	操作当前目录里的内容
`..\images`	先退一层，再进入 `images`	操作旁边目录里的内容
`~\cli-practice`	从自己的主目录找到练习目录	从其他地方快速回到项目

如果电脑有 D 盘，可以用 `cd D:\` 进入 D 盘根目录。需要盘根目录时写清楚反斜杠，不要省略成 `D:`。

记不住路线时就先执行 `pwd`，再用 `ls` 看清目录名。路径只是地址，写出来并不意味着对应的文件夹已经存在。

3. 看懂命令：命令、选项、参数

教程里的命令不总能原样复制：文件名、路径和操作目标经常要换成自己的。学会拆解命令，就能知道哪些部分保留、哪些部分需要替换。

3.1 一条命令的几个部分

为了读懂一行操作，可以先把它分成“做什么”“怎么做”和“对谁做”。常见的入门模型是：

```
命令 选项 参数
```

例如：

```
ls -Force .
```

- **命令** `ls`：做什么？列出目录内容。
- **选项** `-Force`：怎么做？把隐藏项也列出来。
- **参数** `.`：对谁做？当前目录。

各部分通常用空格分隔，输入完按 Enter。选项、参数可以省略，也可能有多个，位置取决于具体命令。例如 `pwd` 不需要目标，`cp 源路径 目标路径` 则需要说明从哪里复制到哪里。

有的选项后面还要跟一个值。看文档时要把“选项的名字”和“给它的值”一起读，不要只复制选项名。

本文在 PowerShell 中执行，选项按本文示例写。例如查看隐藏项用 `ls -Force`，不要直接套用其他环境教程中的 `ls -la`。

3.2 文档里的 `[]`、`<>`、`()` 怎么读？

查文档时，经常会看到带括号的命令写法。要把它变成自己的操作，需要知道哪些部分可以省略、哪些内容需要替换。先看一个例子：

```
demo-tool [--verbose] (show|save) <path>
```

这是**专门用来解释读法的虚构工具**，不要实际执行。这一行是描述用法的“填写模板”，可以从左到右拆开看：

1. 运行名叫 `demo-tool` 的工具。
2. `[--verbose]`：方括号表示这部分可选。想看更详细的输出时写 `--verbose`，不需要时整项省略；不输入方括号。
3. `(show|save)`：这里用圆括号把两个动作分成一组，竖线表示“或”。选择 `show` 或 `save`，不输入圆括号和竖线。
4. `<path>`：尖括号表示占位内容。把它整体换成实际路径，例如 `.` 或 `"C:\My Files"`，不保留尖括号。

因此，`demo-tool show .` 表示选择 `show`，目标为当前目录；`demo-tool --verbose save "C:\My Files"` 则加了可选项，选择 `save`，并提供了另一个路径。

把刚才的读法归纳起来，再补充一个常见的省略号记号：

模板记号	常见含义	输入时怎么处理
`[内容]`	这部分可选	可以不写；要写时去掉外层方括号
`<名称>`	占位内容	用自己的路径、文件名或数值替换整体
`(甲|乙)`	某些文档用它把备选项分为一组	按说明选择一项，不把括号照抄进去
`甲|乙`	“甲或乙”	从备选项中选择
`...`	前面的内容可以重复	可以继续填写同类内容

区分用法模板和实际命令。 实际代码里的 `(1 + 2)` 是表达式，圆括号不能随意删；后面会见到的 `>` 用于把输出写进文件，也不是占位符的尖括号。

3.3 路径有空格，为什么要加引号？

目录名可能是 `My Projects`。直接把它放进命令，空格可能让电脑把一个名字读成两部分。这时用引号把完整路径包起来：

```
cd "C:\My Projects"
```

这是路径示例，需要该目录存在。引号告诉电脑：里面这一整段是一个目标。

输入时可以用 Tab 补全路径，按上下方向键找回历史命令。如果忘了闭合引号，出现了 `>>` 提示，可以按 Ctrl+C 取消后重新输入。

4. 跟着做：创建、查看、复制和删除文件

知道怎么走到一个文件夹后，接下来就要在里面做事。这一节用一个专门的练习目录，把平时用鼠标完成的文件操作串起来。

以下示例按顺序执行。若主目录下已有 `cli-practice` 且存有其他内容，请换一个新的名字，并相应替换后续路径。

4.1 给练习创建一个文件夹

先把练习内容集中放好，避免和自己的其他文件混在一起。这次从确认起点开始，每走一步都核对位置和屏幕上的结果。创建目录用 `mkdir`，进入目录用 `cd`。

下面以用户名 `student` 为例，主目录为 `C:\Users\student`；你的电脑会显示自己的用户名和实际路径。**输入命令时不需要把用户名改成 `student`，`~` 会指向你自己的主目录。**

回显示例只保留需要关注的内容。创建目录、列出目录时，实际屏幕还可能显示权限、修改时间等列，列宽和格式也可能不同。示例中的提示符用于说明位置，不需要输入。

第 1 步：先确认打开窗口后的位置。

刚打开 PowerShell 时，起始位置不一定相同，先不要假定自己在练习目录。输入：

```
pwd
```

预期回显是当前目录的路径。例如，若当前位于用户主目录，会看到：

```
Path
----
C:\Users\student
```

执行前后位置不变。若你看到别的路径，也没关系：先记住它，下一步统一回到主目录。

第 2 步：回到自己的主目录，统一练习起点。

执行前：位于第 1 步查到的目录。输入：

```
cd ~
```

`cd` 成功时通常没有额外回显，默认提示符会变成：

```
PS C:\Users\student>
```

执行后：位于 `C:\Users\student`。再输入 `pwd`，应回显第 1 步示例中的主目录路径；这次无论原先在哪，都应回到自己的主目录。

第 3 步：创建练习目录，但先不进去。

执行前：位于 `C:\Users\student`。输入：

```
mkdir cli-practice
```

预期回显会列出新建的目录，关键内容是：

```
父目录：C:\Users\student
新建目录名：cli-practice
```

上面是对回显关键字段的说明，不是屏幕原文。实际输出通常以表格显示新建目录。

执行后：**仍在 `C:\Users\student`**，提示符仍是 `PS C:\Users\student>`。`mkdir` 只创建文件夹，不会自动进入；如果输入 `pwd`，仍会显示用户主目录。

第 4 步：进入刚创建的练习目录。

执行前：位于 `C:\Users\student`。输入：

```
cd .\cli-practice
```

预期没有额外回显，默认提示符变为：

```
PS C:\Users\student\cli-practice>
```

执行后：位于 `C:\Users\student\cli-practice`。输入 `pwd` 验证，预期回显：

```
Path
----
```

```
C:\Users\student\cli-practice
```

再输入 `ls`。因为这是刚创建的空目录，预期不列出任何文件或子目录，直接出现下一行提示符；当前位置不变。

第 5 步：创建 `notes` 子目录。

执行前：位于 `C:\Users\student\cli-practice`。输入：

```
mkdir notes
```

预期回显列出新建目录 `notes`，其父目录为 `C:\Users\student\cli-practice`。

执行后：仍在 `C:\Users\student\cli-practice`，提示符不变，没有进入 `notes`。

第 6 步：再创建同一层的 `images` 子目录。

执行前：仍位于 `C:\Users\student\cli-practice`。输入：

```
mkdir images
```

预期回显列出新建目录 `images`，其父目录同样为 `C:\Users\student\cli-practice`。执行后位置不变。

接着输入 `ls` 检查，列表中应出现两个目录名，顺序可能不同：

```
images
notes
```

这里仅摘出目录名称。`ls` 不会改变位置，默认提示符仍为 `PS C:\Users\student\cli-practice>`。

第 7 步：进入 `notes`，向下一层。

执行前：位于 `C:\Users\student\cli-practice`。输入：

```
cd .\notes
```

预期没有额外回显，默认提示符变为 `PS C:\Users\student\cli-practice\notes>`。执行后进入 `notes`。输入 `pwd`，预期回显：

```
Path
----
C:\Users\student\cli-practice\notes
```

第 8 步：从 `notes` 去同一层的 `images`。

执行前：位于 `C:\Users\student\cli-practice\notes`。输入：

```
cd ..\images
```

这条路径表示先从 `notes` 回到上一层 `cli-practice`，再进入它下面的 `images`。预期没有额外回显，默认提示符变为 `PS C:\Users\student\cli-practice\images>`。

执行后：位于 `C:\Users\student\cli-practice\images`。输入 `pwd`，预期回显：

```
Path
----
C:\Users\student\cli-practice\images
```

第 9 步：返回上一层，准备后续文件练习。

执行前：位于 `C:\Users\student\cli-practice\images`。输入：

```
cd ..
```

预期没有额外回显，默认提示符变回 `PS C:\Users\student\cli-practice>`。输入 `pwd`，预期回显：

```
Path
----
C:\Users\student\cli-practice
```

再输入 `ls`，仍应看到 `images`、`notes` 两个目录。查看位置和列表都不会移动你。

到这里，**当前位置是 `C:\Users\student\cli-practice`**，目录内容应为：

```
cli-practice\       ← 当前位置
├── images\
└── notes\
```

后面的 4.2 节就从这里继续，不需要重新打开窗口或进入其他目录。

4.2 写一点内容，再读出来

有了目录，还需要创建文件记录内容。可以先用 `echo` 输出文字，再把文字保存到文件中。

```
echo '这是我的命令行练习。'
```

这一行只在屏幕上显示文字。加上 `>`，就把输出写到文件：

```
echo '这是我的命令行练习。' > .\README.txt
cat .\README.txt
```

文件不存在时会创建；已经存在时，`>` 会覆盖原有内容。`cat` 用来查看文件中的文字。

要保留原内容，在末尾增加一行，用两个大于号 `>>`：

```
echo '我会用 pwd 和 ls 了。' >> .\README.txt
cat .\README.txt
```

这时应看到两行：

```
这是我的命令行练习。
我会用 pwd 和 ls 了。
```

记住：`>` 是写入并覆盖，`>>` 是追加。这里的符号要实际输入，不是文档里的占位记号。

本练习在同一 PowerShell 窗口中创建并读取文件。不同版本默认写入的文本编码可能不同。

4.3 复制一份，再移动和改名

需要备份文件或整理位置时，可以用 `cp` 复制、`mv` 移动。这两个命令都按“源路径、目标路径”的顺序填写。

```
cp .\README.txt .\README-copy.txt
mv .\README-copy.txt .\notes\
mv .\notes\README-copy.txt .\notes\lesson.txt
ls .\notes
```

第一行在当前目录复制出一个副本；第二行把副本搬进已经存在的 `notes`；第三行把副本改名为 `lesson.txt`。

原文件 `README.txt` 仍在原处。复制像“再做一份”，移动像“搬走这一份”；同一目录内移动到一个新文件名，就能完成改名。

想复制整个文件夹及里面的内容，可以这样写：

```
cp .\notes .\notes-backup -Recurse
```

`-Recurse` 表示连里面的子目录和文件一起处理。这里假定 `notes-backup` 尚不存在，重复练习前先用 `ls` 看一下状态。

4.4 删除不再需要的副本

练习过程中会留下副本。清理之前，先确认位置和目标，再删除，避免删错自己的文件。

```
pwd
ls .\notes-backup
rm .\notes-backup\lesson.txt -WhatIf
```

`-WhatIf` 只预览将要执行的操作，不真正删除。确认目标是刚创建的备份后，再执行：

```
rm .\notes-backup\lesson.txt
rm .\notes-backup
```

先删除里面的文件，再删除空目录。如果要连目录及内部内容一起删，可以加 `-Recurse`。下面只预览，保留 `notes`：

```
rm .\notes -Recurse -WhatIf
```

`rm` 删除的文件通常不会进入回收站。练习时写清具体路径，不要随意用 `*` 表示一大批文件。

5. 从当前位置打开应用

用命令行找到项目后，不必再回到桌面打开应用、重新找一遍文件夹。把当前位置直接交给应用，就能接着编辑或浏览。

5.1 打开文件夹或指定文件

要让应用直接打开手头的内容，把路径写在应用命令后面即可。保持当前位置为 `cli-practice`，执行：

```
explorer .
notepad .\README.txt
```

第一行用文件资源管理器打开当前文件夹，第二行用记事本打开刚才的文件。

如果某个可执行文件就在当前目录，也可以明确写出位置，例如 `.\hello.exe`。这个例子只有在该文件实际存在时才能执行。

7. 综合练习与速查

单独记住命令还不够，真正使用时需要把“找位置、操作文件、打开应用”连起来。下面用一次小练习检查自己能否独立完成这些动作。

7.1 从创建文件夹到打开项目

这一练习要求你在每次操作前判断当前位置，尤其留意 `.`、`..` 和 `~` 分别指向哪里。

1. 回到自己的主目录，再进入 `cli-practice`。
2. 新建 `exercise`，进入它，写一个包含问候语的 `hello.txt`。
3. 读取文字，把文件复制一份，再把副本改名为 `hello-backup.txt`。
4. 列出上一层的内容，但保持当前位置不变。
5. 用文件资源管理器打开当前目录。
6. 预览删除副本，确认后执行；保留原文件。
7. 返回上一层，确认当前位置。

参考操作，假定 `exercise` 尚不存在：

```
cd ~
cd .\cli-practice
mkdir exercise
cd .\exercise
echo '你好，命令行！' > .\hello.txt
cat .\hello.txt
cp .\hello.txt .\hello-copy.txt
mv .\hello-copy.txt .\hello-backup.txt
ls ..
pwd
explorer .
rm .\hello-backup.txt -WhatIf
rm .\hello-backup.txt
cd ..
pwd
```

完成后，`exercise` 中应保留 `hello.txt`，当前位置应是 `cli-practice`。

7.2 忘了命令时查这里

实际操作中不必一次背住所有写法。先明确自己想做什么，再从下面找到对应命令。表里的路径示例需要根据实际目录替换，不要把整张表当作脚本顺序执行。

想做什么	写法
看自己在哪	`pwd`
看当前目录有什么	`ls` 或 `ls .`
连隐藏项一起看	`ls -Force`
查看上一层，不移动	`ls ..`

想做什么	写法
进入子目录	`cd .\notes`
返回上一层	`cd ..`
返回两层	`cd ..\..`
进入同一层的另一个目录	`cd ..\images`
回到自己的主目录	`cd ~`
创建目录	`mkdir notes`
写入文件，覆盖原内容	`echo '文字' > .\demo.txt`
在文件末尾追加	`echo '更多文字' >> .\demo.txt`
查看文件	`cat .\demo.txt`
复制文件	`cp .\demo.txt .\demo-copy.txt`
移动文件	`mv .\demo-copy.txt .\notes\`
预览删除	`rm .\demo.txt -WhatIf`
实际删除	`rm .\demo.txt`
打开当前文件夹	`explorer .`

7.3 出错时先查位置，再查写法

命令报错时，先根据提示检查最直接的原因，不必马上换工具或重新安装。

现象	优先检查
找不到文件或目录	用 `pwd` 确认位置，用 `ls` 核对名称和目标是否存在
含空格的路径报错	是否用引号包住整个路径
参数无法识别	是否照搬了其他命令行环境的选项，按本文示例核对
提示目录已存在	是否重复做过练习；先看已有内容，不要直接覆盖
输入后一直出现 `>>`	是否忘了闭合引号或括号，Ctrl+C 取消后重输

8. 为什么需要 Git？先用几个例子理解

写作业或做项目时，经常会遇到这些问题：昨天的内容改坏了，想找回来；一周后忘记为什么改了一段代码；和同学各改了一份文件，不知道应该保留哪份。Git 用来管理文件的版本，让修改有记录、能比较，也便于协作。

8.1 从“最终版”文件夹到版本记录

如果只靠复制文件夹保存进度，项目可能变成 作业最终版、作业最终版2、作业真的最终版。这些名字很难说明每一版改了什么。Git 让我们在重要节点主动保存一次记录，并附上一句话说明。

例如，一份作业的历史可以是“完成输入功能”“补充错误提示”“修正计算结果”。每条记录都说明一个具体变化，之后就能查看对应版本，而不必猜哪个文件夹才是想找的那一份。

这里的**提交（commit）**，可以先理解为给选好的项目内容保存一个“存档点”。这是一种帮助理解的类比；提交会记录项目快照、作者、说明等信息，不是额外复制出一个“最终版”文件夹。

8.2 例子一：这次 commit“添加我的介绍”

刚开始学习时，容易把“保存文件”和“提交版本”混为一谈。先用这次的练习仓库看看：我们到底准备提交什么？这一节只讲过程，不用输入命令。

假设你已经把协会的练习项目下载到电脑。里面原本有一份 `README.md`，现在要增加一个以自己名字命名的文件，例如 `xiaoming.txt`。

时刻	你做了什么	Git 里的含义
刚拿到项目	打开原有文件看看	本地已经有项目原来的版本和历史
写完介绍	在 `xiaoming.txt` 写“大家好，我是小明”，保存文件	工作区里多了一个文件，还没有新提交
选好本次内容	决定这次只记录新增的介绍文件	把这个文件的内容放进暂存区，准备提交
正式存档	做一次 commit，说明写“添加小明的介绍”	在已有历史之后增加一条提交记录

这次提交带来的变化是“新增小明的介绍文件”。项目原有内容仍是这个版本的一部分，不会因为只选了新文件就消失。

保存文件，只是把文字写进电脑上的文件；commit 才把选好的内容记录进版本历史。 提交后文件还在原来的目录，不会被搬到别的文件夹。

8.3 例子二：改了两件事，可以分两次 commit

实际操作中，一次可能修改了多个文件，但它们未必属于同一件事。把相关内容放在同一次提交里，更容易说清楚这次改动的目的。

假设小明同时做了两件事：新增自己的介绍文件，还顺手修正了 `README.md` 中的一处错字。他可以这样安排：

1. 先选好介绍文件，做一次 commit，说明是“添加小明的介绍”。
2. 此时 README 的错字修改仍留在电脑上的文件里，但没有进入刚才的提交。
3. 再选好 README 的修改，做另一次 commit，说明是“修正说明中的错字”。

这样回看历史，就能分别找到“加介绍”和“改错字”。**commit 的内容由你事先选好的内容决定，不是自动把电脑上所有修改都存进去。**

这里用到的三个概念可以这样记：

名称	通俗理解	例子
工作区	你正在编辑的项目文件	介绍文件、README 中的当前文字

名称	通俗理解	例子
暂存区	为下一次提交准备好的内容	先选中的介绍文件，不包含尚未选中的错字修改
提交历史	已经正式记录的版本	“添加介绍”“修正错字”这两次提交

这是三个概念，不是要求你自己创建三个文件夹。

8.5 Git、托管平台、本地副本有什么关系？

本次项目已经放在协会的平台上，需要先弄清楚：网页上的仓库和自己电脑里的文件是什么关系。

Git 是版本管理工具；GitHub、协会的 Git 平台等提供仓库托管和协作功能。本次使用的是协会的示例仓库。

按照该仓库 README 的练习流程，我们先 Fork，再 clone：

```
协会平台上的原仓库
BITNP/techdept-git-playground
          | Fork：在平台上创建属于自己账号的仓库副本
          ↓
协会平台上自己账号下的仓库
你的账号/techdept-git-playground
          | clone：把这个仓库克隆到自己的电脑
          ↓
电脑上的 techdept-git-playground 目录
在这里编辑文件、准备内容、做本地提交
```

Fork 完成后，副本仍在平台上；clone 完成后，电脑上才有本地仓库。克隆通常会带下已有的文件和提交历史，不是只下载一包当前文件。

本地 commit 不会自动更新网页上的仓库。 下一节跟着 playground，把本地修改、上传和请求合并串成一次完整练习。

9. 跟着 playground 完成一次协作

下面直接按照协会 playground 的 README 操作：创建 token → Fork → clone → 配置署名 → 添加自己的文件 → 提交并上传 → 创建 PR。目标是让自己写的一份文件经过审核，进入协会的原仓库。

以下假定已经安装 Git，并能登录协会 Git 平台。命令中的用户名、邮箱和文件名换成自己的；回显只展示关键部分，路径、提交编号和进度数字以实际结果为准。

9.1 创建 access token

后面上传改动时，可能需要用 token 完成身份认证，所以先把它准备好。

Access token（访问令牌）可以理解为一种能选择权限的密码。 创建时可以指定它允许做哪些操作，例如读取或写入仓库。当你使用一个令牌登录时，就只能做这个令牌指定允许的操作。后面 Git 要求认证时，可以用它代替账号密码；拿到它的人也能使用这些权限，所以要像保管密码一样保管它。

在协会 Git 平台右上角点击个人头像，按顺序进入：

```
Settings → Applications → New access token
```

按 README 填写：

1. 名字随意，例如 `local`。
2. `Repository and organization access` 选择 `All`。
3. `Select permissions` 中的 `repository` 选择 `Read and write`。
4. 点击 `Generate token`。

生成的 token 会出现在页面上方，复制并妥善保存，后面认证时使用。不要把它写进练习文件或提交到仓库。

9.2 Fork 仓库

为了能上传自己的修改，先在平台上创建一份属于自己的仓库。

打开 BITNP/techdept-git-playground，点击右上角 `Fork`，选择自己的账号并创建。

完成后会进入自己账号下的仓库页面。原仓库属于 `BITNP`，新副本属于你；这时文件还在平台上，下一步才下载到电脑。

9.3 clone 到本地并进入目录

要在电脑上编辑文件，先把自己 Fork 的仓库克隆下来。

打开 PowerShell，进入想存放代码的位置。例如电脑有 D 盘，希望放在 D 盘根目录，就输入：

```
cd D:/
pwd
```

`cd` 成功通常没有文字回显，`pwd` 应显示 `D:\`。没有 D 盘时，可以用 `cd ~` 选择自己的主目录；后续路径相应替换。下面统一以 D 盘为例。

在刚才自己 Fork 的仓库页面复制 HTTPS 克隆地址。命令写法是：

```
git clone <刚才复制的地址>
```

把尖括号及里面的文字替换成实际地址。例如账号是 `yys` 时，地址可能是：

```
git clone https://git.bitnp.net/yys/techdept-git-playground
```

当前位于 `D:\`，预期先显示：

```
Cloning into 'techdept-git-playground'...
```

然后显示下载进度，成功结束后回到提示符。此时仍在 `D:\`，但下面已经多了 `techdept-git-playground` 文件夹。进入它：

```
cd techdept-git-playground
pwd
ls
```

`pwd` 应显示 `D:\techdept-git-playground`，`ls` 应列出仓库中的 `README.md` 等文件。**后面的配置、文件修改、提交和上传，都在这个目录里继续。**

9.4 配置用户名和邮箱

提交需要记录作者的名字和联系方式。这里配置的是提交署名，不是平台的登录密码。

按 README，先设置常用邮箱，再设置英文姓名或网名。当前位于 `D:\techdept-git-playground`，把下面的示例值换成自己的：

```
git config --global user.email "you@example.com"
git config --global user.name "Your Name"
```

成功时通常没有回显，当前位置不变。`--global` 表示为当前电脑上的这个用户设置默认署名，通常只需要配置一次；已经正确配置过就不必重复。按本次练习要求使用英文名或网名，不必实名。

9.5 加入自己的改动

这次要提交的内容很简单：新增一个以自己名字命名的文件，写一些自己想说的话。

仍在 `D:\techdept-git-playground`，例如创建 `yys.txt`。可以用记事本打开：

```
notepad .\yys.txt
```

写一段自己的话，保存到当前项目目录，编码选择 UTF-8，再关闭记事本。文件名换成自己的名字，避免覆盖其他人的文件。

这一步保存了文件，但还没有提交。PowerShell 的当前位置仍是 `D:\techdept-git-playground`。

9.6 提交改动，再上传到自己的仓库

文件写好后，先把修改记录成本地提交，再上传到刚才 Fork 的仓库。这是两个连续但不同的动作。

第一步：准备提交内容。 在项目目录执行 README 中的命令：

```
git add .
```

成功时通常没有回显。`.` 表示当前目录，这里会暂存项目目录及其子目录中的变化；因此下一步要检查是否只包含自己想提交的内容。

第二步：检查暂存结果。 当前位置不变，输入：

```
git status
```

如果本次只新增了 `yys.txt`，预期关键回显是：

```
Changes to be committed:
        new file:   yys.txt
```

这表示介绍文件已经准备好，可以提交。文件名应是你自己的文件；如果还列出了不打算提交的修改，先核对清楚。

第三步：创建提交。 继续在项目目录输入：

```
git commit -m "add yys' file"
```

`-m` 后面是这次改动的说明，换成自己的名字和内容。预期回显类似：

```
[main abc1234] add yys' file
 1 file changed, 1 insertion(+)
 create mode 100644 yys.txt
```

这表示本地新增了一次提交，`abc1234` 是示例编号。此时网页上的仓库还没有更新，继续上传：

第四步：push 到自己的 Fork。 当前位置仍是项目目录，执行：

```
git push origin main
```

按 README，这份仓库的远程名称是 `origin`，分支是 `main`；`origin` 指向刚才克隆的、自己账号下的仓库。

如果出现下面的认证提示：

```
Username for 'https://git.bitnp.net':
Password for 'https://yys@git.bitnp.net':
```

`Username` 填通行证用户名，`Password` 填第 9.1 节保存的 token。也可能直接打开浏览器，出现 `Authorize "Git Credential Manager" to access your account?`，按 README 点击 `Authorize application` 即可。

上传成功的关键回显类似：

```
To https://git.bitnp.net/yys/techdept-git-playground
   4ec8a9a..e0b92f9  main -> main
```

账号和提交编号以实际结果为准。命令执行后当前位置不变。刷新自己 Fork 的仓库页面，应能看到刚添加的文件。

9.7 创建 PR，请维护者合并

现在改动已经在自己的仓库里。要让它进入协会原仓库，还需要发起一个合并请求，也就是 Pull Request（PR）。

打开原仓库 BITNP/techdept-git-playground，进入 `Pull Requests`，点击 `New Pull Request`。

按 README 选择两端：

页面字段	应选择什么
`merge into`：要合并到哪里	原仓库 `BITNP:main`
`pull from`：改动来自哪里	自己 Fork 的仓库，例如 `yys:main`

检查页面显示的改动是否符合预期，写好标题和说明，点击 `Create Pull Request`。

创建 PR 后，等待维护者审核并合并。**发出请求不等于已经合并**；合并完成后，刷新原仓库，才能在那里看到自己的改动。

如果维护者要求修改，就继续在本地这个项目的 `main` 分支修改文件，再按刚才的顺序 add、检查 status、commit、push。按 README，向同一个来源分支继续推送后，尚未合并的 PR 会随之更新，不需要重新创建一份。

参考资料

需要核对具体行为时，可以查看以下官方说明；阅读正文不要求先掌握这些文档中的完整命令名称。

- 路径与当前位置
- 文档中的命令语法记号
- 输出重定向：`>` 与 `>>`
- 不同版本的文本编码

- Git：版本与快照
- Git：首次配置
- Git：创建与克隆仓库
- Git：记录修改
- 协会 Git 练习仓库：练习流程依据用户提供的 README，在线页面此前未能读取。